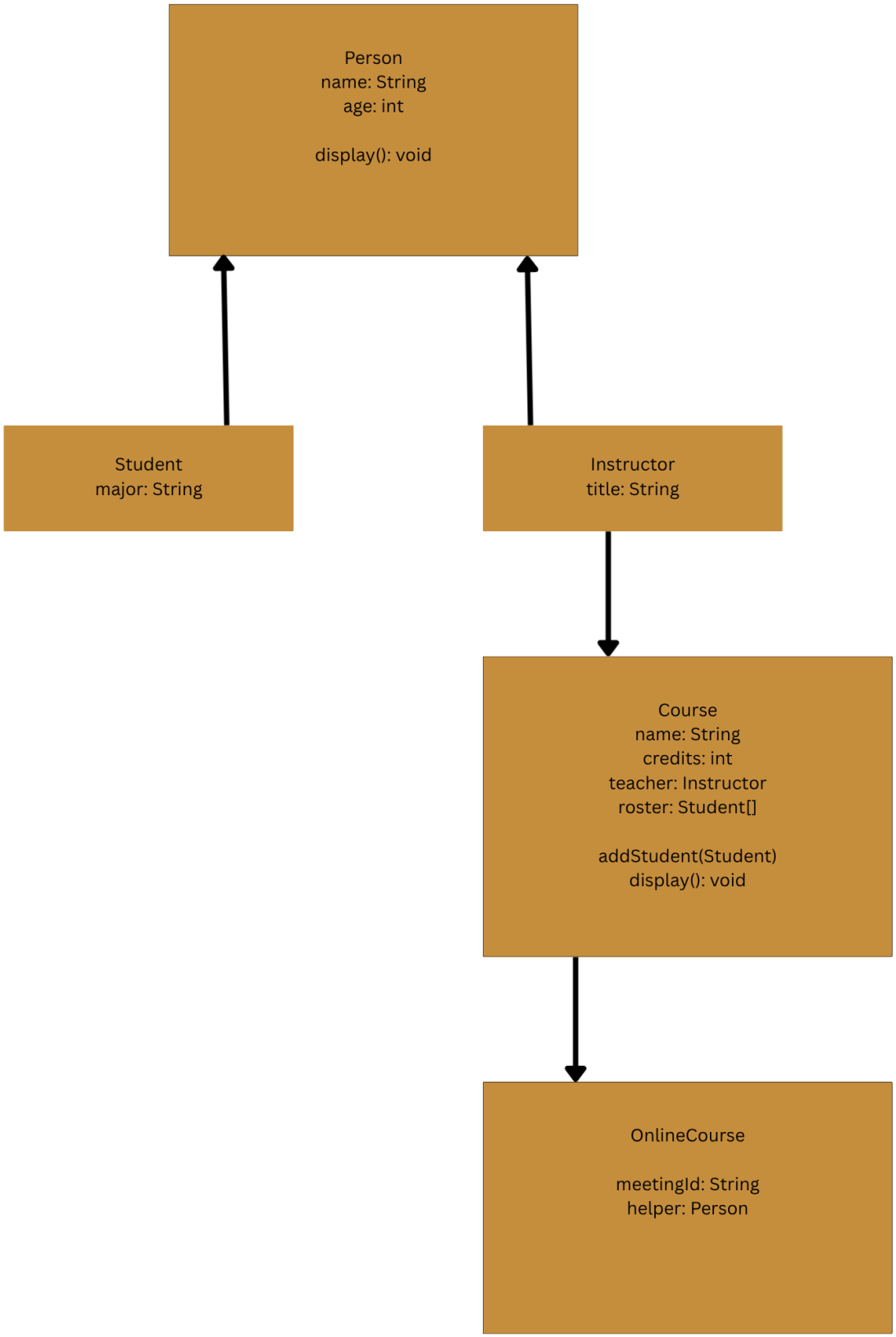
1) Include the docstrings documentation

```
class Queue:
    """
    This class implements a first in first out (FIFO) data structure using
    two lists: one for enqueuing elements and another for dequeuing elements.
    """

    def __init__(self):
        """
        Initialize an empty Queue with two internal lists.

        Attributes:
        a_in (list): Internal list that stores newly enqueued elements.
        a_out (list): Internal list that stores elements ready to be dequeued.
        """
        self.a_in = []
        self.a_out = []


    def enqueue(self, d):
        """
        Add an element to the back of the queue.

        Args:
        d: The element to add to the queue. Can be of any type.
        """
        self.a_in.append(d)

    def dequeue(self):
        """
        Remove and return the element at the front of the queue.

        Returns:
            The element at the front of the queue (the oldest element).
        """
        if (self.a_out == []):
            for d in self.a_in:
                self.a_out.append(d)
            self.a_in = []
        return self.a_out.pop(0)
```

2. Convert from python to Java

```java
import java.util.Random;
import java.util.Scanner;
/**
 * A Rock Scissors Paper game.
 */
public class RSP {
    private static final String ROCK = "r";
    private static final String SCISSORS = "s";
    private static final String PAPER = "p";

    /**
     * Main method to run the Rock Scissors Paper game.
     *
     * @param args command line arguments (not used)
     */
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String playerChoice = "";

        while (true) {
            System.out.print("Enter (r)ock, (s)cissors, or (p)aper: ");
            playerChoice = scanner.nextLine().toLowerCase().trim();

            if (playerChoice.equals(ROCK) || playerChoice.equals(SCISSORS) ||
playerChoice.equals(PAPER)) {
                break;
            }
            System.out.println("Try again.");
        }

        String[] choices = {ROCK, SCISSORS, PAPER};
        String computerChoice = choices[new Random().nextInt(3)];

        if (playerChoice.equals(computerChoice)) {
            System.out.println("It's a tie!");
        } else if ((playerChoice.equals(ROCK) && computerChoice.equals(SCISSORS)) ||
                   (playerChoice.equals(PAPER) && computerChoice.equals(ROCK)) ||
                   (playerChoice.equals(SCISSORS) && computerChoice.equals(PAPER))) {
            System.out.println("You win! Computer chose " + computerChoice);
        } else {
            System.out.println("You lose! Computer chose " + computerChoice);
        }

        scanner.close();
    }
}
```

3) **Class Diagram**



4) Java Collections

A **Stack collection** is last in first out implementation, consider a stack of plates you would take from the top, meaning the most recently added item is removed first.
A **Queue collection** is first in first out implementation, consider a line of people to get into a concert, you take the first person in line.

5) SDLC

The **advantages** of having the SDLC is that it provides a **structured way to test software** before putting out the wild, it should go through the **wringer.** SDLC will **ensure** that all necessary steps are followed and that the final product meets the requirements and acceptance criteria. It will help to **identify any potential issues** early in the development process, reducing the risk and bad reputation.

6) Version Control

I use a Mac, so I ran these following commands. Since it's a virtual environment I don't have to change anything in my python files, dependencies are isolated. These commands setup the environment, install the deps and save it to a text file that I can call later.

```
**python3 -m venv env**
**source env/bin/activate**
**pip install numpy==1.18.5 xlwt==1.3.0**
```

To deactivate it run `**deactivate**`

To call it again `**source env/bin/activate**`

7) Profiling

Before running the code at face value, **the check function** looks to be doing the most its **O(n^2)** because of the nested loops. But what's calling it is the layer function. **Layer** is doing a depth first search over all possible matrices and check is getting called every time counting each 0 and 1 in every column. So, **backtracking** would be my **real solution** keeping track of what was already counted instead of starting over each time. But I'd try to **optimize the layer function.**

8) SDM

Waterfall is defined as a simple approach that **is not flexible** enough for projects that need adjustments along the way.
Agile is not the answer because it is considered the most **flexible** and is made for catching mistakes and errors in the early stages.
DevOps is not the answer because it is a mix of both lean and agile so not this one which means it's **flexible.**
Lean is the answer which has a strategy of "don't do today what can be done tomorrow" which means **it's not flexible** enough for changes during the project which is **Waterfall.**